

# Gestión profesional del contexto en ChatGPT para proyectos largos

## El modelo mental correcto: contexto, historial, memoria y documentación no son lo mismo

La idea más importante es esta: **ChatGPT no posee una memoria humana continua de todo lo que has dicho**. En cada respuesta trabaja con una colección limitada de información preparada para ese turno. Esa colección puede contener partes de la conversación, instrucciones, memorias, archivos, resultados de herramientas y otros elementos, pero no debe confundirse con el historial completo que ves en la interfaz.

El modelo mental profesional separa cuatro capas:

| Capa                            | Qué representa                                                                 | Para qué sirve                                    | Nivel de fiabilidad                                                              |
|---------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Historial almacenado</b>     | Los mensajes que aparecen en la interfaz y permanecen asociados al chat        | Consultar conversaciones anteriores y reabrir las | Puede estar almacenado sin estar íntegramente disponible para el modelo          |
| <b>Contexto activo</b>          | La información que el modelo puede utilizar al generar la siguiente respuesta  | Razonar sobre la tarea actual                     | Limitado por la ventana de contexto y por cómo la aplicación prepara la petición |
| <b>Memoria de ChatGPT</b>       | Información seleccionada de conversaciones, archivos o preferencias anteriores | Personalización y continuidad entre chats         | Útil, pero no exhaustiva ni adecuada como base de datos exacta                   |
| <b>Fuente de verdad externa</b> | Repositorio, documentación, especificaciones, decisiones, pruebas y registros  | Conservar el estado real del proyecto             | Debe ser la capa canónica de un proyecto profesional                             |

La regla práctica que se deriva de esta separación es:

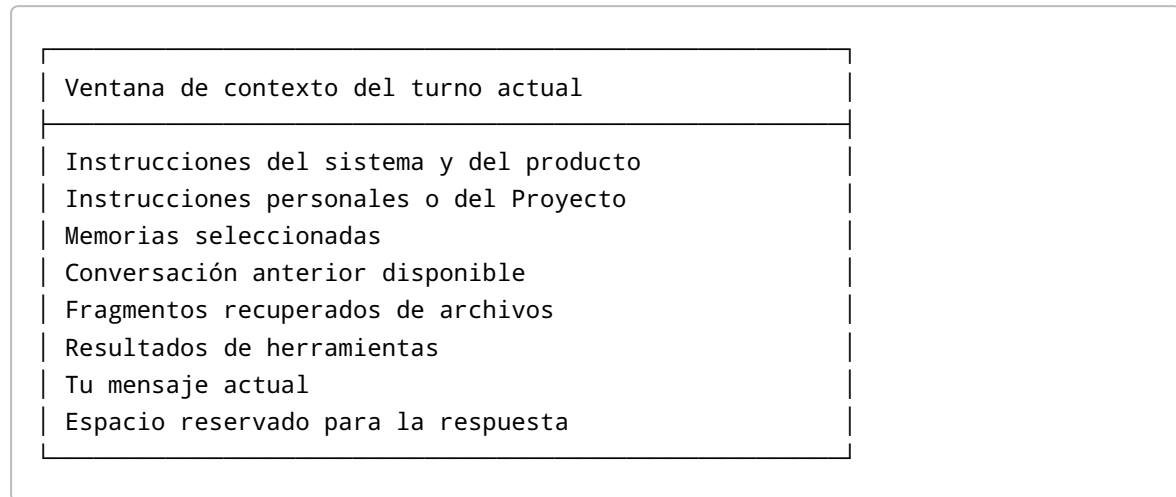
**El chat sirve para pensar y trabajar; los archivos sirven para recordar con precisión.**

### Qué es exactamente la ventana de contexto

La ventana de contexto es la cantidad máxima de información, medida en **tokens**, que un modelo puede considerar en una generación. Los tokens no corresponden exactamente a palabras: pueden ser palabras completas, fragmentos, signos de puntuación o caracteres. OpenAI define el límite como una

capacidad combinada que debe alojar la entrada y la salida generada; el tamaño exacto depende del modelo y del producto utilizado. <sup>1</sup>

Conceptualmente, puede imaginarse así:

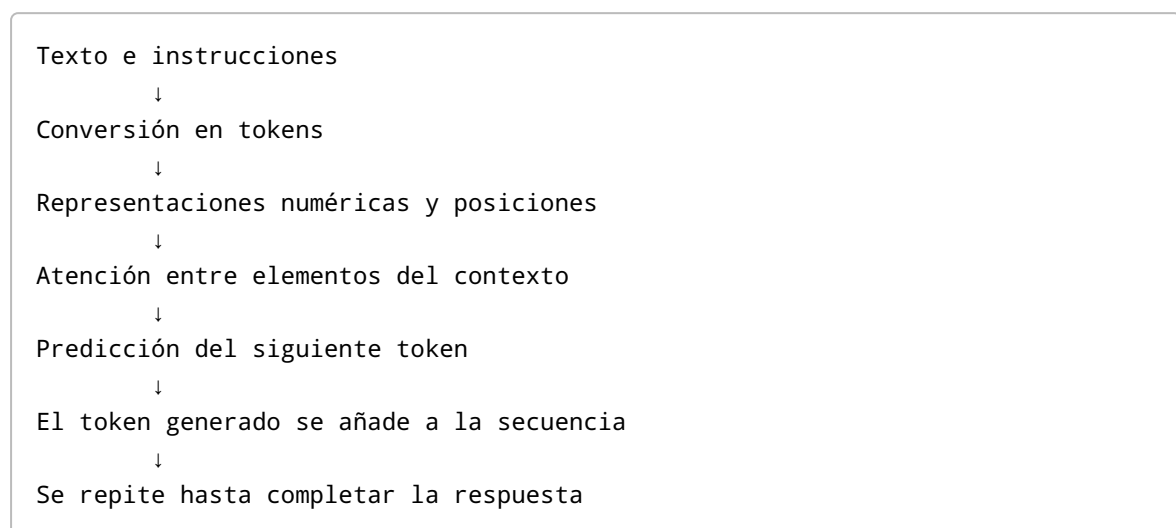


Todo lo que entre en esa caja compite por espacio. Una conversación extensa no solo consume contexto con tus mensajes: también lo hacen las respuestas del asistente, los fragmentos de código, los errores de terminal, los documentos, las definiciones de herramientas y la futura respuesta. En las interfaces de programación de OpenAI, cada solicitud de generación es conceptualmente independiente; para conservar continuidad hay que proporcionar el estado anterior directamente o utilizar mecanismos de gestión de conversaciones. <sup>2</sup>

## Cómo se utiliza internamente a nivel conceptual

Los modelos modernos se basan en la arquitectura Transformer. Su mecanismo de atención calcula relaciones entre posiciones de una secuencia y permite que la generación tenga en cuenta diferentes partes del contexto. No almacena recuerdos como una persona: transforma los tokens disponibles en representaciones matemáticas y genera la respuesta progresivamente. <sup>3</sup>

Una simplificación útil es:



Esto explica por qué una información puede estar técnicamente presente y, aun así, no influir correctamente en la respuesta. **“Estar dentro de la ventana” no garantiza “ser recuperado, interpretado y aplicado correctamente”.**

Los experimentos sobre contextos largos han encontrado que algunos modelos recuperan peor la información colocada en la parte central de una entrada extensa que aquella situada cerca del principio o del final, fenómeno conocido como *lost in the middle*. También se han observado sesgos de posición y degradación cuando el razonamiento requiere combinar muchas piezas dispersas. <sup>4</sup>

Por eso los desarrolladores distinguen entre:

- **Capacidad máxima de contexto:** cuántos tokens caben técnicamente.
- **Contexto efectivo:** cuánta información puede utilizar el modelo con suficiente fiabilidad para la tarea concreta.
- **Contexto relevante:** la pequeña parte que realmente necesita para resolver el siguiente paso.

Una ventana enorme no elimina la necesidad de seleccionar, estructurar y resumir. La documentación oficial de OpenAI señala que el rendimiento de contextos largos puede empeorar cuando aumenta el número de elementos que deben recuperarse o cuando la tarea exige mantener el estado global de toda la entrada. También recomienda reordenar instrucciones, explicitar restricciones y volver a “anclar” el modelo en tareas densas. <sup>5</sup>

## Qué ocurre dentro de una conversación larga

### ¿Lee ChatGPT todo el chat cada vez?

La respuesta precisa es: **el modelo utiliza el contexto que ChatGPT prepara para ese turno, no necesariamente una copia literal e íntegra de todo lo que aparece en la pantalla.**

En una conversación corta, ese contexto puede aproximarse mucho al intercambio completo. Cuando la conversación, los archivos o los resultados de herramientas se vuelven extensos, el producto puede necesitar seleccionar, recuperar, resumir o compactar información. OpenAI documenta mecanismos de compactación para sus interfaces de programación que reducen el tamaño del contexto conservando el estado considerado necesario; sin embargo, no publica todos los detalles internos de cómo la interfaz de ChatGPT representa cada conversación en cada turno. <sup>6</sup>

Por tanto, no debes razonar así:

“El mensaje sigue visible, por lo que el modelo siempre lo tiene presente.”

Debes razonar así:

“El mensaje sigue almacenado, pero debo asegurarme de que las decisiones críticas estén reintroducidas, recuperadas o documentadas.”

Un chat puede conservar cien páginas visibles y, aun así, la respuesta actual estar condicionada principalmente por instrucciones, mensajes recientes, resúmenes, memorias o fragmentos considerados relevantes.

## Las distintas formas de “olvidar”

Cuando se dice que ChatGPT “ha olvidado” algo, pueden estar ocurriendo fenómenos diferentes:

| Tipo de fallo                 | Qué ha ocurrido                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------|
| <b>Exclusión</b>              | La información ya no está incluida en el contexto activo                          |
| <b>Compresión con pérdida</b> | La conversación se resumió, pero se perdió un matiz, número o excepción           |
| <b>Fallo de recuperación</b>  | La información existe en un archivo o chat, pero no se seleccionó para este turno |
| <b>Fallo de atención</b>      | Está presente, pero el modelo no le asignó suficiente relevancia                  |
| <b>Conflicto</b>              | Existen instrucciones o decisiones incompatibles y el modelo eligió una           |
| <b>Obsolescencia</b>          | Una afirmación antigua compite con una decisión posterior                         |
| <b>Inferencia errónea</b>     | El modelo conserva los hechos, pero extrae una conclusión incorrecta              |

Esta distinción es muy importante al depurar una conversación. Repetir “ya te lo dije” no soluciona necesariamente el problema. En su lugar, conviene identificar qué dato falta, cuál es su fuente canónica y qué instrucción debe aplicarse ahora.

## Qué información suele degradarse antes

No existe un orden universal y determinista. No puede asegurarse que ChatGPT elimine siempre “lo más antiguo primero”. Sin embargo, son especialmente vulnerables:

1. **Detalles antiguos que no se han vuelto a mencionar**, sobre todo si están enterrados en una conversación extensa.
2. **Restricciones pequeñas dentro de mensajes largos**, como una excepción, una fecha exacta o una condición negativa.
3. **Decisiones provisionales**, porque el modelo puede no distinguirlas de decisiones definitivas.
4. **Información situada entre grandes bloques de código, registros o documentos**, debido a la competencia por atención.
5. **Datos contradictorios**, especialmente cuando no se indica cuál es la versión vigente.
6. **Relaciones que requieren combinar varios mensajes distantes**, por ejemplo una regla mencionada al principio y una excepción introducida cincuenta turnos después.
7. **Resultados extensos de herramientas**, como miles de líneas de terminal, pruebas o volcados de bases de datos.

Los estudios de contextos largos muestran que la posición, la distancia entre evidencias y la cantidad de información irrelevante afectan a la recuperación y al razonamiento. No basta con que la información quepa dentro del límite nominal. <sup>7</sup>

## Contexto del chat frente a memoria permanente

El **contexto del chat** es la información utilizada para producir la respuesta actual. Es temporal y tiene capacidad limitada.

La **memoria de ChatGPT** es una capa de personalización que puede conservar información útil procedente de chats, archivos y aplicaciones conectadas. OpenAI permite consultar y editar un resumen de memoria, pero advierte que ese resumen no contiene necesariamente todo lo que ChatGPT ha aprendido o puede utilizar. <sup>8</sup>

La memoria es apropiada para cosas como:

```
Estoy aprendiendo programación.  
Prefiero explicaciones en español de España.  
Trabajo principalmente con TypeScript y Python.  
Quiero que señales los riesgos y no me des la razón automáticamente.
```

No es una buena ubicación para:

```
El endpoint /api/v2/invoices devuelve 409 cuando el campo currency  
no coincide con la cuenta, excepto para migraciones anteriores al  
commit a73f92c.
```

Ese segundo tipo de información debe estar en el código, las pruebas, la documentación o un registro de decisiones.

La memoria tampoco debe entenderse como una base de datos transaccional. Está diseñada para aportar personalización y continuidad, no para garantizar la recuperación exacta de cada requisito, versión, número o decisión. <sup>9</sup>

## Cómo organizar chats y resúmenes sin perder continuidad

### Un único chat enorme frente a varios chats especializados

Para proyectos de meses, el mejor patrón general no es elegir exclusivamente uno de los dos extremos. Es utilizar una jerarquía:

```
Un Proyecto de ChatGPT por producto o iniciativa  
|  
├─ Un chat de visión y requisitos  
├─ Un chat de arquitectura  
├─ Un chat por funcionalidad importante  
├─ Chats de depuración temporales  
├─ Un chat de aprendizaje  
└─ Archivos canónicos compartidos por todos
```

Un chat único tiene la ventaja de conservar continuidad narrativa. El modelo conoce cómo se llegó a una decisión y puedes utilizar referencias como “la alternativa que descartamos ayer”. Funciona bien durante una fase breve y cohesionada: diseñar un módulo, depurar un error concreto o preparar una migración.

Su inconveniente es que acumula ruido: opciones descartadas, errores resueltos, código antiguo, hipótesis falsas y largas respuestas que ya no son relevantes. Conforme crece, aumenta el riesgo de contradicciones, recuperación deficiente y respuestas condicionadas por estados obsoletos.

Los chats especializados reducen el ruido y facilitan que cada conversación tenga una misión clara. El inconveniente es que una conversación nueva no recibe automáticamente todos los razonamientos exactos de otra. Este problema se soluciona usando instrucciones de proyecto, archivos canónicos y un resumen de traspaso.

La decisión profesional puede expresarse así:

| Situación                                   | Enfoque recomendado                                        |
|---------------------------------------------|------------------------------------------------------------|
| Una sesión de depuración de una o dos horas | Continuar en el mismo chat                                 |
| Desarrollo de una funcionalidad completa    | Un chat dedicado a la funcionalidad                        |
| Decisión arquitectónica transversal         | Chat de arquitectura y registro posterior en un ADR        |
| Preguntas para aprender un concepto         | Chat de aprendizaje separado                               |
| Investigación de una tecnología alternativa | Chat separado para evitar contaminar el contexto principal |
| Proyecto con varios meses de duración       | Proyecto de ChatGPT con múltiples chats y documentación    |
| Conversación que mezcla cinco temas         | Separarla inmediatamente                                   |

## Cuándo merece la pena abrir un chat nuevo

Abre un chat nuevo cuando cambie la **unidad de trabajo**, no simplemente porque haya pasado tiempo.

Las señales más claras son:

- Pasas de analizar requisitos a implementar.
- Cambias de una funcionalidad a otra independiente.
- Empiezas una investigación que podría producir mucho contenido descartable.
- La depuración va a generar numerosos registros, pruebas y callejones sin salida.
- La conversación contiene muchas decisiones obsoletas.
- Necesitas cambiar el papel del asistente: arquitecto, revisor, profesor o depurador.
- ChatGPT empieza a mezclar nombres, versiones, requisitos o archivos.
- Te cuesta explicar en una frase cuál es el objetivo actual del chat.

Una excelente prueba es completar esta frase:

“Este chat existe exclusivamente para...”

Cuando no puedas terminarla con una misión concreta, el contexto ya está demasiado mezclado.

## Cuándo pedir un resumen

No tienes que esperar a que la ventana esté completamente llena. Es mejor crear puntos de control antes de que aparezca una degradación severa.

Pide un resumen cuando:

- Se termina una fase: diseño, implementación, pruebas o lanzamiento.
- Se ha tomado una decisión importante.
- Vas a continuar en otro chat o dispositivo.
- La conversación contiene múltiples intentos fallidos.
- Has modificado requisitos durante la sesión.
- ChatGPT repite preguntas ya resueltas.
- Aparecen respuestas genéricas que ignoran restricciones específicas.
- Confunde el estado actual con una propuesta antigua.
- Recomienda una solución que ya se descartó.
- Empieza a inventar nombres de archivos, funciones o componentes.
- Necesitas releer muchos mensajes para saber qué hacer después.

La guía de OpenAI para tareas de contexto largo recomienda volver a explicitar las restricciones, generar esquemas de las secciones relevantes y anclar las afirmaciones a fuentes concretas. Esa técnica reduce los errores de pérdida dentro de entradas densas. <sup>10</sup>

## Cómo debe ser un buen resumen de continuidad

Un resumen útil no es una narración cronológica de la conversación. Debe representar el **estado operativo actual**.

Este es un buen prompt de traspaso:

Prepara un resumen de continuidad para comenzar un chat nuevo.

No narres toda la conversación. Conserva únicamente el estado vigente.

Incluye:

- Objetivo del proyecto.
- Objetivo de la tarea actual.
- Estado actual verificable.
- Decisiones definitivas y su justificación.
- Alternativas descartadas y por qué.
- Restricciones funcionales y técnicas.
- Arquitectura y componentes afectados.
- Archivos, clases, endpoints y tablas relevantes.
- Comandos de instalación, ejecución y pruebas.
- Trabajo completado.
- Problemas conocidos.
- Preguntas abiertas.
- Próximo paso exacto.
- Criterio de aceptación de la tarea.

- Fechas, cifras e identificadores que deban conservarse literalmente.

Señala explícitamente cualquier dato del que no estés seguro.  
No conviertas hipótesis en hechos.

La información que nunca debería faltar es:

| Elemento                        | Por qué es imprescindible                                 |
|---------------------------------|-----------------------------------------------------------|
| <b>Objetivo actual</b>          | Evita que el nuevo chat optimice otra cosa                |
| <b>Estado real</b>              | Distingue lo implementado de lo simplemente propuesto     |
| <b>Decisiones vigentes</b>      | Evita reabrir debates cerrados                            |
| <b>Restricciones</b>            | Impide soluciones técnicamente válidas pero incompatibles |
| <b>Fuente canónica</b>          | Permite comprobar la información                          |
| <b>Próxima acción</b>           | Elimina el coste de reconstruir el punto de continuación  |
| <b>Preguntas abiertas</b>       | Evita que el modelo rellene huecos inventando             |
| <b>Criterio de finalización</b> | Permite saber cuándo la tarea está realmente terminada    |

Después de recibir el resumen, revísalo manualmente. El resumen es otra salida probabilística del modelo: puede omitir precisamente el detalle que querías preservar. Para información crítica, copia los valores exactos desde la fuente original y añádelos tú mismo.

## Cómo trabajan los profesionales en proyectos que duran meses

Los desarrolladores que gestionan bien proyectos con IA no suelen tratar una conversación como un compañero que recordará todo durante meses. Utilizan una arquitectura de información parecida a la de un equipo humano:

|                             |                                 |
|-----------------------------|---------------------------------|
| Repositorio y documentación | = memoria institucional         |
| Issues y planificación      | = trabajo pendiente             |
| Commits y pruebas           | = evidencia de lo realizado     |
| Chats                       | = sesiones de trabajo           |
| Resúmenes                   | = traspasos entre sesiones      |
| Instrucciones               | = reglas operativas             |
| IA                          | = lector y colaborador temporal |

El patrón robusto es **persistir el conocimiento en artefactos legibles tanto por humanos como por la IA**.

### Estructura documental recomendada

Para cada aplicación, mantendría una estructura parecida a esta:



```

mi-aplicacion/
├── README.md
├── docs/
│   ├── vision.md
│   ├── requirements.md
│   ├── architecture.md
│   ├── current-state.md
│   ├── roadmap.md
│   ├── glossary.md
│   ├── testing-strategy.md
│   ├── security.md
│   ├── runbook.md
│   └── decisions/
│       ├── ADR-001-database.md
│       ├── ADR-002-authentication.md
│       └── ADR-003-event-processing.md
├── CHANGELOG.md
├── CONTRIBUTING.md
└── src/

```

Cada archivo tiene una responsabilidad diferente:

- `vision.md` explica qué problema resuelve el producto y para quién.
- `requirements.md` contiene requisitos funcionales y no funcionales.
- `architecture.md` describe componentes, límites y flujos.
- `current-state.md` representa dónde está el proyecto ahora.
- `roadmap.md` muestra las fases previstas, sin sustituir a un gestor de tareas.
- Los ADR registran decisiones arquitectónicas, alternativas y consecuencias.
- `runbook.md` explica cómo ejecutar, desplegar, restaurar y diagnosticar.
- Las pruebas representan requisitos ejecutables.

La documentación oficial de Claude Code recomienda invertir en documentación y memoria de proyecto precisamente para que el agente pueda comprender el repositorio sin tener que reconstruir su funcionamiento en cada sesión. <sup>11</sup>

## Un chat por funcionalidad, no un chat por archivo

Separar cada fichero en un chat diferente suele fragmentar demasiado el razonamiento. La unidad adecuada es una **funcionalidad o problema con un criterio de aceptación**.

Por ejemplo, para una aplicación SaaS:

```

Chat: Arquitectura general y límites del sistema
Chat: Registro e inicio de sesión
Chat: Recuperación de contraseña
Chat: Suscripciones y facturación
Chat: Panel de administración

```

Chat: Depuración del error de sincronización  
Chat: Revisión de seguridad antes del lanzamiento

El chat de facturación puede necesitar modificar diez archivos, pero todos pertenecen a una misma unidad conceptual. Cuando la funcionalidad termina, se actualizan la documentación, las pruebas y el estado del proyecto; después se inicia otra conversación.

## Protocolo profesional de una sesión

Al comenzar una sesión:

Lee primero:

- docs/current-state.md
- docs/architecture.md
- docs/decisions/ADR-003-event-processing.md

Objetivo de esta sesión:

Implementar reintentos idempotentes para los webhooks de facturación.

Antes de modificar código:

1. Resume el estado relevante.
2. Indica qué archivos necesitas inspeccionar.
3. Propón un plan verificable.
4. Señala cualquier contradicción en la documentación.

Durante la sesión, trabaja en incrementos pequeños. Tras una decisión importante, actualiza el documento correspondiente. Tras implementar, ejecuta pruebas. Tras corregir una suposición falsa, registra el descubrimiento si puede ser útil en sesiones futuras.

Al cerrar:

Antes de terminar:

1. Resume qué se ha implementado realmente.
2. Enumera las pruebas ejecutadas y sus resultados.
3. Distingue trabajo completado, parcial y no iniciado.
4. Actualiza current-state.md.
5. Propón las modificaciones necesarias en los ADR.
6. Prepara el siguiente paso en una sola tarea.
7. Genera un resumen para un chat nuevo.

Este enfoque se parece a los flujos recomendados por las herramientas de programación con IA: explorar primero, planificar antes de editar, delegar investigaciones y mantener el contexto principal centrado en la tarea. <sup>12</sup>

## Por qué no conviene mantener un único chat durante meses

Un único chat puede parecer cómodo porque “lo contiene todo”, pero a largo plazo mezcla:

- Requisitos antiguos y nuevos.
- Código previo y actual.
- Decisiones provisionales y definitivas.
- Errores ya resueltos.
- Salidas de herramientas enormes.
- Explicaciones educativas que no son necesarias para implementar.
- Hipótesis que más tarde se demostraron falsas.

La conversación termina convirtiéndose en un registro histórico, no en un contexto de trabajo eficiente. Un registro histórico puede ser útil para humanos, pero es una mala entrada operativa si no se filtra.

El objetivo profesional no es maximizar la cantidad de contexto. Es maximizar la proporción:

información relevante / información total

## Qué cambia dentro de un Proyecto de ChatGPT

A fecha de agosto de 2026, OpenAI describe los Proyectos como espacios de trabajo que agrupan conversaciones, archivos, fuentes e instrucciones para esfuerzos prolongados. Las instrucciones del Proyecto se aplican dentro de él y sustituyen a las instrucciones personalizadas globales cuando existe conflicto. <sup>13</sup>

### Ventajas frente a un chat normal

Un Proyecto ofrece una capa organizativa por encima de los chats:

| Chat normal                                      | Proyecto de ChatGPT                             |
|--------------------------------------------------|-------------------------------------------------|
| Una conversación principal                       | Múltiples conversaciones relacionadas           |
| Archivos asociados principalmente al intercambio | Fuentes reutilizables en el ámbito del Proyecto |
| Instrucciones generales de la cuenta             | Instrucciones específicas del Proyecto          |
| Continuidad centrada en el chat                  | Memoria y recuperación orientadas al Proyecto   |
| Difícil separar iniciativas                      | Frontera conceptual por producto o iniciativa   |

OpenAI indica que los chats de un Proyecto pueden aprovechar los archivos e instrucciones del mismo y que, en determinados planes y configuraciones de memoria, pueden hacer referencia a otras conversaciones de ese Proyecto. ChatGPT prioriza los chats y archivos del Proyecto al responder dentro de él. <sup>14</sup>

Esto permite una estructura adecuada para proyectos grandes:

Proyecto: Aplicación de gestión financiera

Fuentes:

- visión del producto
- requisitos
- arquitectura
- decisiones
- esquema de base de datos
- plan de pruebas
- estado actual

Chats:

- descubrimiento
- arquitectura
- autenticación
- importación bancaria
- categorización con IA
- facturación
- despliegue
- aprendizaje y dudas

## Memoria exclusiva del Proyecto

Al crear un Proyecto puede estar disponible la opción de memoria exclusiva del Proyecto. En ese modo, los chats pueden utilizar conversaciones del mismo Proyecto, pero no conversaciones externas ni memorias guardadas fuera de él. En proyectos compartidos se establecen límites de contexto para impedir que se mezclen memorias personales ajenas al Proyecto. <sup>15</sup>

Esta opción es especialmente útil cuando gestionas proyectos muy distintos:

Proyecto A: aplicación médica  
Proyecto B: videojuego  
Proyecto C: curso de Python  
Proyecto D: gestión personal

Sin una frontera clara, una preferencia o supuesto de un proyecto podría influir indebidamente en otro. La memoria exclusiva actúa como aislamiento conceptual, aunque no sustituye la documentación precisa.

## El papel de las instrucciones del Proyecto

Las instrucciones deben describir **reglas estables de colaboración**, no toda la historia del producto.

Un ejemplo adecuado:

Actúa como arquitecto y mentor de programación.

Contexto:

Estoy aprendiendo, pero quiero utilizar prácticas profesionales.

Reglas:

- Explica las decisiones importantes y sus alternativas.
- No inventes el contenido de archivos que no hayas leído.
- Distingue hechos, hipótesis y recomendaciones.
- Antes de cambiar arquitectura, consulta los ADR.
- Propón cambios pequeños y verificables.
- Incluye pruebas para cada modificación funcional.
- No declares una tarea terminada sin indicar qué se ha verificado.
- Prioriza TypeScript estricto, claridad y mantenibilidad.
- Cuando falte información, identifica exactamente qué archivo o dato hace falta.

No sería adecuado colocar ahí cientos de líneas sobre la estructura de cada tabla, todos los endpoints o cada error conocido. Las instrucciones permanentes compiten por atención en cada sesión; deben ser concisas, coherentes y verificables. OpenAI recomienda organizar claramente las instrucciones largas y revisar contradicciones o reglas insuficientemente especificadas. <sup>16</sup>

## Qué debe guardarse en archivos

Utiliza archivos para información que deba ser:

- Exacta.
- Reutilizable.
- Versionable.
- Revisable.
- Compartible.
- Auditable.
- Independiente de una conversación concreta.

Esto incluye:

| Guardar en archivo                   | No depender únicamente del historial         |
|--------------------------------------|----------------------------------------------|
| Requisitos y criterios de aceptación | “Lo comentamos hace tres semanas”            |
| Arquitectura vigente                 | Razonamiento disperso en varios chats        |
| Contratos de API y esquemas          | Respuestas generadas en una sesión           |
| Decisiones y consecuencias           | Memoria informal del asistente               |
| Comandos de ejecución y despliegue   | Una explicación antigua                      |
| Problemas conocidos                  | Un mensaje enterrado                         |
| Plan de pruebas                      | Una promesa de que “debería funcionar”       |
| Estado actual                        | El último punto que ChatGPT parezca recordar |
| Glosario de dominio                  | Definiciones implícitas                      |
| Fechas, cifras y versiones           | Memoria personalizada                        |

ChatGPT permite además guardar una respuesta valiosa como fuente del Proyecto. Esto puede utilizarse para conservar un resumen, una decisión o un análisis, aunque conviene convertir posteriormente la información crítica en documentación mantenida y versionada. <sup>17</sup>

## El Proyecto no sustituye al repositorio

El Proyecto debe considerarse un **espacio de colaboración con IA**, no el sistema oficial de control del producto.

Para una aplicación real:

|                     |                                                      |
|---------------------|------------------------------------------------------|
| Git y repositorio   | → fuente de verdad del código                        |
| Pruebas             | → fuente de verdad del comportamiento esperado       |
| Documentación       | → fuente de verdad del diseño y operación            |
| Gestor de tareas    | → fuente de verdad del trabajo pendiente             |
| Proyecto de ChatGPT | → espacio para analizar, aprender y producir cambios |

Si ChatGPT afirma que una función existe y el repositorio muestra que no existe, gana el repositorio. Si un resumen dice que las pruebas pasaron y el sistema de integración continua muestra un fallo, gana la integración continua.

## Comparación con Claude Code: el contexto como recurso gestionado

ChatGPT y Claude Code utilizan modelos con ventanas de contexto, pero su unidad de trabajo es diferente.

ChatGPT es principalmente una experiencia **centrada en conversaciones y Proyectos**. Claude Code es una herramienta **centrada en un repositorio y su entorno de desarrollo**: puede inspeccionar archivos, editar código, ejecutar comandos y trabajar con herramientas del sistema. Anthropic describe Claude Code como un agente de programación que opera directamente sobre el código y las herramientas del proyecto. <sup>18</sup>

### Diferencias conceptuales

| Dimensión             | ChatGPT con Proyecto                                | Claude Code                                                    |
|-----------------------|-----------------------------------------------------|----------------------------------------------------------------|
| Centro del trabajo    | Chats, fuentes e instrucciones                      | Repositorio, terminal, archivos y herramientas                 |
| Inicio de una sesión  | Puede utilizar memoria y contexto del Proyecto      | Cada sesión empieza con una ventana nueva                      |
| Persistencia          | Memoria, historial, fuentes del Proyecto            | <code>CLAUDE.md</code> , reglas, memoria automática y archivos |
| Obtención de contexto | Conversaciones, fuentes y recuperación del producto | Lectura activa de archivos y búsquedas en el repositorio       |

| Dimensión             | ChatGPT con Proyecto                                   | Claude Code                                             |
|-----------------------|--------------------------------------------------------|---------------------------------------------------------|
| Especialización       | Chats separados e instrucciones de Proyecto            | Skills, subagentes, reglas por rutas                    |
| Reducción de contexto | Gestión interna, resúmenes y compactación del producto | Compactación automática y comando <code>/compact</code> |
| Fuente canónica ideal | Archivos y repositorio externos                        | El propio repositorio y su documentación                |

Anthropic explica explícitamente que cada sesión de Claude Code comienza con una ventana de contexto nueva. El conocimiento entre sesiones se conserva mediante archivos `CLAUDE.md` escritos por el usuario y una memoria automática en la que Claude registra aprendizajes, correcciones o patrones del repositorio. <sup>19</sup>

### Cómo ayuda `CLAUDE.md`

`CLAUDE.md` es un archivo de instrucciones persistentes que Claude Code carga al comenzar una sesión. Puede contener:

```
# Comandos

- Instalar: `pnpm install`
- Desarrollo: `pnpm dev`
- Pruebas: `pnpm test`
- Tipos: `pnpm typecheck`

# Arquitectura

- Los controladores HTTP están en `src/api/handlers`.
- La lógica de dominio no puede depender de Express.
- Las consultas a la base de datos pasan por repositorios.

# Convenciones

- Utilizar TypeScript estricto.
- No usar `any` sin justificación.
- Añadir pruebas para cada corrección.
- No modificar migraciones ya desplegadas.

# Flujo de trabajo

- Leer el ADR correspondiente antes de cambiar arquitectura.
- Ejecutar pruebas y comprobación de tipos antes de finalizar.
```

Anthropic recomienda mantener estos archivos concisos, específicos y bien estructurados; su documentación propone como objetivo menos de unas doscientas líneas por archivo. El contenido se carga en la ventana y consume contexto, por lo que un `CLAUDE.md` enorme puede reducir la adherencia a las instrucciones. <sup>19</sup>

La lección aplicable a ChatGPT es directa: **las instrucciones globales deben contener reglas permanentes, no una enciclopedia del proyecto.**

## Cómo ayudan las Skills

Una Skill encapsula un procedimiento reutilizable, por ejemplo:

```
/revisar-seguridad  
/crear-migracion  
/depurar-webhook  
/preparar-release  
/revisar-pull-request
```

La diferencia importante es que el cuerpo de una Skill se carga cuando se utiliza, en lugar de ocupar permanentemente el contexto de cada sesión. Anthropic recomienda convertir en Skill las instrucciones repetitivas, listas de comprobación y procedimientos de varios pasos que no necesitan estar presentes en todo momento. <sup>20</sup>

Ejemplo de división eficiente:

```
CLAUDE.md  
└─ Regla permanente: "Toda migración debe ser reversible."  
  
Skill: crear-migracion  
├─ Cómo nombrarla  
├─ Cómo generar el archivo  
├─ Cómo probar subida y bajada  
├─ Cómo comprobar compatibilidad  
└─ Lista de verificación antes del commit
```

La regla breve permanece siempre. El procedimiento detallado solo entra en contexto cuando se crea una migración.

En ChatGPT puedes imitar este patrón guardando procedimientos en archivos separados dentro del Proyecto:

```
skills/  
├─ code-review.md  
├─ database-migration.md  
├─ release-checklist.md  
└─ incident-debugging.md
```

Al iniciar una tarea, indicas a ChatGPT que utilice únicamente el procedimiento correspondiente.

## Cómo ayudan los subagentes

Los subagentes ejecutan tareas en contextos separados. Un agente principal puede delegar:



Subagente A → localizar la implementación de autenticación  
Subagente B → revisar las pruebas existentes  
Subagente C → analizar posibles riesgos de seguridad  
Agente principal → integrar los resultados y decidir

Anthropic señala que los subagentes permiten conservar el contexto principal evitando introducir en él toda la exploración y la implementación secundaria. También pueden limitar herramientas, utilizar instrucciones especializadas y elegir modelos diferentes según el coste o la tarea. <sup>21</sup>

Esto reduce el problema típico de una sesión de programación:

Objetivo original:  
Corregir la validación de una factura.

Contexto contaminante:

- 300 resultados de búsqueda
- 10 archivos irrelevantes
- 4 hipótesis descartadas
- 800 líneas de pruebas
- registros completos del servidor

El subagente puede consumir ese ruido y devolver algo compacto:

La validación está en `src/billing/validateInvoice.ts`.  
La llamada principal procede de `src/api/invoices/create.ts`.  
Existen cuatro pruebas relacionadas.  
El error se debe a que `currency` se valida después de persistir.  
Recomiendo mover la validación antes de `repository.save()`.

## Cómo ayuda la compactación

Claude Code puede compactar automáticamente una conversación cuando se aproxima al límite de contexto. La compactación convierte partes anteriores en una representación más pequeña para continuar trabajando. La configuración de auto-compactación está activada por defecto y puede ajustarse; también existe una operación manual. <sup>22</sup>

OpenAI proporciona un concepto parecido en su API: la compactación conserva estado anterior utilizando menos tokens y permite continuar interacciones prolongadas equilibrando calidad, coste y latencia. <sup>6</sup>

La compactación no es mágica. Puede preservar:

"Se decidió utilizar PostgreSQL."

y perder:

“Se eligió PostgreSQL 17 porque necesitamos restricciones diferibles para el proceso de conciliación, pero el entorno local aún utiliza PostgreSQL 16 y requiere una ruta alternativa.”

Por eso la información crítica debe residir en documentos exactos, no exclusivamente en el resumen generado por la compactación.

## La mayor diferencia no es el tamaño de la ventana

Claude puede ofrecer ventanas muy grandes según el modelo, pero su ventaja operativa para programación procede principalmente de una gestión explícita del contexto:

- Inicio limpio por sesión.
- Repositorio como fuente de información.
- Instrucciones persistentes y concisas.
- Carga bajo demanda de Skills.
- Reglas limitadas por rutas.
- Subagentes con contextos aislados.
- Compactación automática.
- Documentación accesible desde el mismo entorno.

Anthropic advierte que más contexto no equivale automáticamente a mejores resultados y denomina *context rot* a la degradación de precisión y recuperación a medida que aumenta la entrada. Su recomendación es seleccionar lo que entra en contexto, no limitarse a aumentar su capacidad. <sup>23</sup>

## Sistema profesional para tus próximos doce meses

Si fuera tu mentor, organizaría tu trabajo alrededor de un principio:

**Cada proyecto debe poder sobrevivir a la desaparición de todos sus chats.**

Si perdieras hoy las conversaciones, deberías ser capaz de reconstruir el proyecto leyendo el repositorio, la documentación, las pruebas y el gestor de tareas. Los chats acelerarían el trabajo, pero no serían imprescindibles para conocer la realidad.

## Arquitectura general de tu espacio de trabajo

Crearía tres niveles:

### Nivel personal

- └─ Proyecto ChatGPT: Aprendizaje de programación e IA
- └─ Archivo: objetivos-anuales.md
- └─ Archivo: mapa-de-competencias.md
- └─ Archivo: diario-de-aprendizaje.md

### Nivel de producto

- └─ Un Proyecto de ChatGPT por aplicación
- └─ Un repositorio Git por aplicación

- └ Documentación canónica
- └ Gestor de tareas

Nivel de sesión

- └ Un chat por funcionalidad o problema
- └ Un objetivo verificable
- └ Archivos relevantes
- └ Resumen de cierre

No mezclaría el aprendizaje general con el contexto de implementación. Por ejemplo:

Proyecto de aprendizaje:

"Explicame qué es la inversión de dependencias."

Proyecto de la aplicación:

"Aplica inversión de dependencias al servicio de facturación siguiendo architecture.md y ADR-006."

El primero prioriza comprensión. El segundo prioriza coherencia con el producto.

## Tipos de chats que utilizaría

Dentro de cada Proyecto mantendría pocos tipos reconocibles:

| Tipo de chat         | Propósito                                         | Duración habitual                         |
|----------------------|---------------------------------------------------|-------------------------------------------|
| <b>Visión</b>        | Producto, usuarios, alcance y prioridades         | Larga, pero con pocos mensajes            |
| <b>Arquitectura</b>  | Decisiones transversales y ADR                    | Reutilizable con puntos de control        |
| <b>Funcionalidad</b> | Implementar una capacidad concreta                | Hasta completar el criterio de aceptación |
| <b>Depuración</b>    | Investigar un fallo específico                    | Temporal                                  |
| <b>Revisión</b>      | Seguridad, pruebas, calidad o rendimiento         | Una revisión determinada                  |
| <b>Aprendizaje</b>   | Comprender conceptos surgidos durante el proyecto | Separado de la implementación             |

Usaría nombres que indiquen claramente su estado:

[ARQ] Autenticación y sesiones  
 [FEATURE] Importación de movimientos bancarios  
 [BUG] Duplicación de webhooks Stripe  
 [REVIEW] Seguridad previa a beta  
 [LEARNING] Transacciones e idempotencia

Cuando una conversación termine, actualizaría su título:

```
[DONE] Importación de movimientos bancarios
[ARCHIVED] Investigación de Firebase descartada
[BLOCKED] Integración bancaria pendiente de credenciales
```

## Rutina de inicio de cada proyecto

Antes de escribir código, produciría cinco documentos pequeños:

```
docs/vision.md
docs/requirements.md
docs/architecture.md
docs/current-state.md
docs/roadmap.md
```

Después crearía las instrucciones del Proyecto. Finalmente abriría tres chats iniciales:

```
Visión y alcance
Arquitectura inicial
Primer incremento funcional
```

No intentaría diseñar todo el producto en una única conversación. El primer objetivo sería construir un recorrido vertical pequeño:

```
Interfaz → API → lógica → base de datos → prueba → despliegue
```

Esto te permite aprender el ciclo completo antes de añadir complejidad.

## Rutina diaria

Cada sesión empezaría con este contrato:

```
Contexto canónico:
- docs/current-state.md
- docs/architecture.md
- ADR relevante
- issue actual

Objetivo:
[una sola tarea]

Criterio de aceptación:
[condiciones observables]

Antes de actuar:
- Comprueba el estado.
```

- Señala contradicciones.
- Propón un plan pequeño.
- No supongas el contenido de archivos no inspeccionados.

Durante la sesión:

Explorar → planificar → cambiar → probar → revisar → documentar

Al final:

- Actualizar `current-state.md`.
- Registrar decisiones permanentes.
- Guardar comandos o hallazgos reutilizables.
- Crear o actualizar pruebas.
- Dejar un próximo paso exacto.
- Generar un resumen de continuidad si la tarea continúa.

## Rutina semanal

Una vez por semana realizaría una revisión de treinta a sesenta minutos:

| Revisión       | Pregunta                                                |
|----------------|---------------------------------------------------------|
| Estado         | ¿ <code>current-state.md</code> representa la realidad? |
| Requisitos     | ¿Ha cambiado algo sin documentarlo?                     |
| Arquitectura   | ¿Se ha tomado una decisión que necesita ADR?            |
| Contexto       | ¿Hay chats que mezclan demasiados temas?                |
| Instrucciones  | ¿Existen reglas obsoletas o contradictorias?            |
| Calidad        | ¿Pasan las pruebas y comprobaciones?                    |
| Aprendizaje    | ¿Qué conceptos todavía no comprendes?                   |
| Próxima semana | ¿Cuál es el siguiente incremento demostrable?           |

Crearía además un punto de control semanal:

```
# Estado semanal

## Entregado
- ...

## Decisiones
- ...

## Riesgos
```

```
- ...

## Deuda técnica
- ...

## Aprendizajes
- ...

## Próximo objetivo
- ...
```

Ese documento permite abrir un chat nuevo sin depender del historial anterior.

## Rutina mensual

Cada mes haría una revisión más profunda:

- Eliminar instrucciones duplicadas.
- Comprobar que los ADR siguen siendo vigentes.
- Archivar chats terminados.
- Dividir documentos demasiado grandes.
- Revisar dependencias y versiones.
- Revisar seguridad y copias de seguridad.
- Comprobar que el repositorio puede instalarse desde cero.
- Evaluar qué tareas delegaste bien a la IA y cuáles aceptaste sin comprender.
- Seleccionar dos o tres conceptos técnicos para estudiar deliberadamente.

También pediría a ChatGPT una auditoría, pero obligándolo a basarse en fuentes concretas:

```
Revisa el proyecto usando únicamente:
- requirements.md
- architecture.md
- current-state.md
- los ADR
- las pruebas

Busca:
- contradicciones
- requisitos sin pruebas
- decisiones no documentadas
- documentación obsoleta
- riesgos técnicos
- áreas donde el código no coincide con la arquitectura

Cita el archivo y sección correspondiente.
Distingue evidencia de inferencia.
```

## Evolución durante el año

**Durante los primeros tres meses**, priorizaría fundamentos: Git, terminal, pruebas, HTTP, bases de datos, tipos, depuración y documentación. Construirías una aplicación pequeña, pero completa. El objetivo no sería producir mucho código, sino aprender a verificar cada afirmación de la IA.

**Entre el cuarto y el sexto mes**, desarrollarías una aplicación mayor organizada por funcionalidades. Introducirías ADR, integración continua, pruebas de integración, despliegues repetibles y revisiones de seguridad. Cada funcionalidad tendría su propio chat y criterio de aceptación.

**Entre el séptimo y el noveno mes**, trabajarías en varios proyectos, pero limitarías el trabajo activo a uno principal y uno secundario. Empezarías a crear procedimientos reutilizables: plantillas de revisión, depuración, migraciones y lanzamiento. Si utilizas Claude Code, transformarías esos procedimientos en Skills.

**Entre el décimo y el duodécimo mes**, profesionalizarías el sistema: observabilidad, rendimiento, seguridad, recuperación ante fallos, documentación para colaboradores y automatización de controles. Revisarías proyectos antiguos para comprobar si otra persona —o una sesión nueva de IA— puede comprenderlos sin leer tus chats.

## Buenas prácticas esenciales

Las prácticas que más reducirán la pérdida de contexto son:

1. **Un Proyecto de ChatGPT por aplicación o dominio estable.**
2. **Un chat por funcionalidad, incidente o decisión importante.**
3. **Documentos canónicos fuera del flujo conversacional.**
4. **Instrucciones breves, estables y sin contradicciones.**
5. **Resúmenes orientados al estado, no a la historia.**
6. **Decisiones registradas con fecha, justificación y consecuencias.**
7. **Pruebas como evidencia, no como promesa.**
8. **Reanclaje explícito al comenzar y terminar cada sesión.**
9. **Revisión humana de cualquier resumen crítico.**
10. **Separación entre aprender un concepto e implementar una tarea.**

## Errores habituales de principiantes

**Tratar el chat como una base de datos.** Que una conversación conserve un dato no garantiza que sea recuperado correctamente meses después. La solución es escribirlo en una fuente canónica.

**Mantener un solo chat para todo.** Parece cómodo al principio, pero termina mezclando producto, programación, aprendizaje, errores y decisiones antiguas.

**Copiar demasiado contexto “por seguridad”.** Añadir todo el repositorio, todos los registros y toda la documentación puede reducir la claridad. Primero hay que identificar qué información es relevante.

**No distinguir propuesta de decisión.** Frases como “podríamos usar Redis” pueden reaparecer después como si Redis hubiese sido elegido. Etiqueta explícitamente:

Estado: propuesta  
Estado: decisión aprobada  
Estado: descartado  
Estado: pendiente de validación

**Cambiar requisitos solo dentro del chat.** Si una regla del producto cambia, actualiza `requirements.md` y las pruebas correspondientes.

**Guardar procedimientos enormes en instrucciones permanentes.** Las instrucciones deben ser reglas constantes; los procedimientos detallados deben cargarse cuando se necesiten.

**Pedir resúmenes narrativos.** Un resumen de veinte párrafos sobre “lo que hablamos” es menos útil que un estado estructurado con decisiones, archivos, riesgos y próximo paso.

**Aceptar código que no entiendes.** La continuidad del proyecto depende de que tú puedas diagnosticarlo sin la conversación que lo generó.

**No verificar el contenido del repositorio.** El asistente puede recordar una versión antigua de una función. Lee el archivo real antes de diseñar el siguiente cambio.

**No cerrar las sesiones.** Abandonar un chat sin actualizar estado, pruebas y documentación hace que el siguiente inicio requiera reconstruir el proyecto desde mensajes dispersos.

## La regla final

El flujo profesional no consiste en conseguir que ChatGPT recuerde todo. Consiste en diseñar un sistema en el que **no necesite recordar todo**.

La memoria sostenible de un proyecto debe estar distribuida así:

|                              |                                 |
|------------------------------|---------------------------------|
| Preferencias personales      | → memoria de ChatGPT            |
| Reglas del producto          | → instrucciones del Proyecto    |
| Conocimiento exacto          | → archivos y repositorio        |
| Procedimientos reutilizables | → documentos o Skills           |
| Exploración temporal         | → chats especializados          |
| Continuidad entre sesiones   | → resumen de traspaso           |
| Comportamiento verificable   | → pruebas                       |
| Historia de cambios          | → Git y registros de decisiones |

Cuando trabajas de esta manera, la ventana de contexto deja de ser una limitación misteriosa. Se convierte en lo que es para los desarrolladores experimentados: **un presupuesto de memoria de trabajo que debe reservarse para la decisión que estás tomando ahora, mientras el conocimiento duradero permanece fuera del modelo, estructurado, verificable y bajo tu control.**

---

<sup>1</sup> What are tokens and how to count them? | OpenAI Help Center  
<https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>



- 2 Conversation state | OpenAI API  
<https://developers.openai.com/api/docs/guides/conversation-state>
- 3 Attention Is All You Need  
[https://arxiv.org/abs/1706.03762?utm\\_source=chatgpt.com](https://arxiv.org/abs/1706.03762?utm_source=chatgpt.com)
- 4 7 Lost in the Middle: How Language Models Use Long Contexts  
[https://arxiv.org/abs/2307.03172?utm\\_source=chatgpt.com](https://arxiv.org/abs/2307.03172?utm_source=chatgpt.com)
- 5 16 GPT-4.1 Prompting Guide  
[https://developers.openai.com/cookbook/examples/gpt4-1\\_prompting\\_guide](https://developers.openai.com/cookbook/examples/gpt4-1_prompting_guide)
- 6 Compaction | OpenAI API  
<https://developers.openai.com/api/docs/guides/compaction>
- 8 9 Memory FAQ | OpenAI Help Center  
<https://help.openai.com/articles/8590148-memory-faq>
- 10 GPT-5.2 Prompting Guide  
[https://developers.openai.com/cookbook/examples/gpt-5/gpt-5-2\\_prompting\\_guide](https://developers.openai.com/cookbook/examples/gpt-5/gpt-5-2_prompting_guide)
- 11 Enterprise deployment overview - Claude Code Docs  
[https://docs.anthropic.com/en/docs/claude-code/enterprise-setup?utm\\_source=chatgpt.com](https://docs.anthropic.com/en/docs/claude-code/enterprise-setup?utm_source=chatgpt.com)
- 12 Common workflows - Claude Code Docs  
<https://docs.anthropic.com/en/docs/claude-code/common-workflows>
- 13 14 15 17 Projects in ChatGPT | OpenAI Help Center  
<https://help.openai.com/en/articles/10169521-projects-in-chatgpt>
- 18 Overview - Claude Code Docs  
[https://docs.anthropic.com/en/docs/claude-code/overview?utm\\_source=chatgpt.com](https://docs.anthropic.com/en/docs/claude-code/overview?utm_source=chatgpt.com)
- 19 How Claude remembers your project - Claude Code Docs  
<https://docs.anthropic.com/en/docs/claude-code/memory>
- 20 Extend Claude with skills - Claude Code Docs  
<https://docs.anthropic.com/en/docs/claude-code/skills>
- 21 Create custom subagents - Claude Code Docs  
<https://docs.anthropic.com/en/docs/claude-code/sub-agents>
- 22 Claude Code settings - Claude Code Docs  
<https://docs.anthropic.com/en/docs/claude-code/settings>
- 23 Context windows - Claude Platform Docs  
<https://docs.anthropic.com/en/docs/build-with-claude/context-windows>